



Universidad Nacional Autónoma de México



Facultad de Ingeniería

Practica 4:

Filtro pasa bajas, filtro pasa altas, filtro pasa bandas-
Pico a Matlab.

Integrantes:

Espinoza Matamoros Percival Ulises - 320025561

Flores Colin Victor Jaziel - 320266083

García Cortés Adolfo de Jesús - 320317264

Lara Hernandez Angel Husiel - 320060829

Lugo Manzano Rodrigo - 320206968

Microcomputadoras

Grupo: 03 - Semestre: 2026-2

Calf: _____



1. Objetivo:

Diseñar e implementar un simulador de funciones de transferencia discretas de primer y segundo orden mediante una Raspberry Pi Pico 2W programada en MicroPython, capaz de aplicar filtros IIR (pasa bajas, pasa altas y pasa banda) en tiempo real a una señal de entrada muestreada a 8 kHz, comunicándose con MATLAB a través de un enlace Bluetooth HC-05 para la recepción de coeficientes y el envío de datos de entrada/salida para su visualización y análisis espectral.

2. Desarrollo:

Parámetros iniciales:

- Frecuencia de muestreo: $F_s = 8000$ Hz
- Periodo de muestreo: $T = \frac{1}{F_s} = \frac{1}{8000} = 125 \mu s$
- Frecuencia de corte pasa bajas: $f_c = 85$ Hz
- Frecuencia de corte pasa altas: $f_c = 480$ Hz
- Frecuencia central (pasa banda): $f_o = 580$ Hz
- Ancho de banda (pasa banda): $BW = 45$ Hz
- Número de muestras por captura: 500
- Tipo de filtro: Butterworth IIR

2.1. Diseño del Filtro Pasa Bajas de Primer Orden

El prototipo analógico Butterworth de primer orden tiene la función de transferencia:

$$H(s) = \frac{\omega_c}{s + \omega_c} \quad (1)$$

Se calcula la frecuencia de corte pre-distorsionada para compensar el efecto de la transformación bilineal:



$$\Omega_c = 2F_s \tan\left(\frac{\pi f_c}{F_s}\right) = 2(8000) \tan\left(\frac{\pi \cdot 85}{8000}\right) = 16000 \tan(0.03337) \quad (2)$$

$$\Omega_c = 16000 \times 0.033386 = 534.18 \text{ rad/s} \quad (3)$$

Aplicación de la Transformación Bilineal

Sustituyendo $s = 2F_s \cdot \frac{z-1}{z+1}$ en $H(s)$ con ω_c reemplazado por Ω_c :

$$H(z) = \frac{\Omega_c}{2F_s \cdot \frac{z-1}{z+1} + \Omega_c} \quad (4)$$

Multiplicando numerador y denominador por $(z+1)$:

$$H(z) = \frac{\Omega_c(z+1)}{2F_s(z-1) + \Omega_c(z+1)} \quad (5)$$

Expandiendo el denominador:

$$H(z) = \frac{\Omega_c(z+1)}{(2F_s + \Omega_c)z + (\Omega_c - 2F_s)} \quad (6)$$

Dividiendo todo entre $(2F_s + \Omega_c)$:

$$H(z) = \frac{\frac{\Omega_c}{2F_s + \Omega_c}(z+1)}{z - \frac{2F_s - \Omega_c}{2F_s + \Omega_c}} \quad (7)$$

Cálculo de los Coeficientes

$$A_0 = A_1 = \frac{\Omega_c}{2F_s + \Omega_c} = \frac{534.18}{16000 + 534.18} = \frac{534.18}{16534.18} \approx 0.03230 \quad (8)$$

$$A_2 = 0 \quad (9)$$

$$B_1 = \frac{2F_s - \Omega_c}{2F_s + \Omega_c} = \frac{16000 - 534.18}{16534.18} = \frac{15465.82}{16534.18} \approx 0.93540 \quad (10)$$

$$B_2 = 0 \quad (11)$$

Ganancia en DC

En DC ($z = 1$):

$$H(1) = \frac{A_0 + A_1}{1 - B_1} = \frac{0.03230 + 0.03230}{1 - 0.93540} = \frac{0.06460}{0.06460} = 1.0 \checkmark \quad (12)$$

Esto confirma que el filtro tiene ganancia unitaria en DC, como se espera de un pasa bajas.

2.1.1. Función de Transferencia:

$$H(z) = \frac{0.03230 z + 0.03230}{z - 0.93540} = \frac{0.03230(1 + z^{-1})}{1 - 0.93540 z^{-1}} \quad (13)$$

Diagrama de Bloques

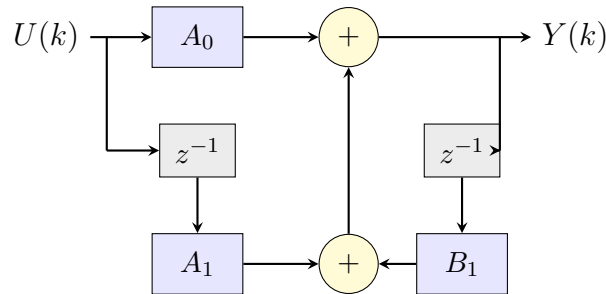


Figura 1: Diagrama de bloques del filtro pasa bajas de primer orden (Forma Directa I).

2.1.2. Frecuencias de entradas en matlab:

Tabla 1: Frecuencias de prueba para el filtro pasa bajas ($f_c = 85$ Hz)

Frecuencia	Relación con f_c	Comportamiento esperado
20 Hz	$f \ll f_c$	Paso sin atenuación
50 Hz	$f < f_c$	Paso con mínima atenuación, curva exponencial visible
85 Hz	$f = f_c$	Atenuación de -3 dB (70.7 %)
200 Hz	$f > f_c$	Atenuación significativa
500 Hz	$f \gg f_c$	Atenuación fuerte



2.2. Diseño del Filtro Pasa Altas de Primer Orden

El prototipo analógico Butterworth pasa altas de primer orden:

$$H(s) = \frac{s}{s + \omega_c} \quad (14)$$

Para el filtro pasa altas se selecciona $f_c = 480$ Hz. La frecuencia pre-distorsionada es:

$$\Omega_c = 2F_s \tan\left(\frac{\pi \cdot 480}{8000}\right) = 16000 \tan(0.18850) = 16000 \times 0.19089 = 3054.24 \text{ rad/s} \quad (15)$$

Sustituyendo en la transformación bilineal:

$$H(z) = \frac{2F_s \cdot \frac{z-1}{z+1}}{2F_s \cdot \frac{z-1}{z+1} + \Omega_c} \quad (16)$$

Multiplicando por $(z + 1)$:

$$H(z) = \frac{2F_s(z - 1)}{2F_s(z - 1) + \Omega_c(z + 1)} = \frac{2F_s(z - 1)}{(2F_s + \Omega_c)z + (\Omega_c - 2F_s)} \quad (17)$$

Dividiendo entre $(2F_s + \Omega_c)$:

$$H(z) = \frac{\frac{2F_s}{2F_s + \Omega_c}(z - 1)}{z - \frac{2F_s - \Omega_c}{2F_s + \Omega_c}} \quad (18)$$

Cálculo de los Coeficientes

$$A_0 = \frac{2F_s}{2F_s + \Omega_c} = \frac{16000}{19054.24} \approx 0.83970 \quad (19)$$

$$A_1 = -A_0 = -0.83970 \quad (20)$$

$$A_2 = 0 \quad (21)$$

$$B_1 = \frac{2F_s - \Omega_c}{2F_s + \Omega_c} = 0.67940 \quad (22)$$

$$B_2 = 0 \quad (23)$$

Ganancia en DC y Nyquist

En DC ($z = 1$):

$$H(1) = \frac{A_0 + A_1}{1 - B_1} = \frac{0.83970 - 0.83970}{1 - 0.67940} = \frac{0}{0.32060} = 0 \checkmark \quad (24)$$

En Nyquist ($z = -1$):

$$H(-1) = \frac{A_0(-1) + A_1}{-1 - (-B_1)} = \frac{-0.83970 - 0.83970}{-1 - 0.67940} = \frac{-1.67940}{-1.67940} = 1.0 \checkmark \quad (25)$$

Función de Transferencia Final

$$H(z) = \frac{0.83970z - 0.83970}{z - 0.67940} = \frac{0.83970(1 - z^{-1})}{1 - 0.67940z^{-1}} \quad (26)$$

Diagrama de Bloques

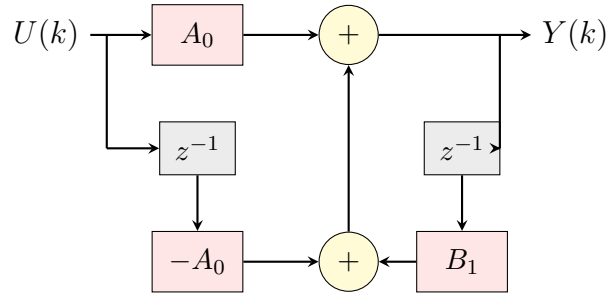


Figura 2: Diagrama de bloques del filtro pasa altas de primer orden (Forma Directa I).

Frecuencias de entradas en matlab:

Tabla 2: Frecuencias de prueba para el filtro pasa altas ($f_c = 480$ Hz)

Frecuencia	Relación con f_c	Comportamiento esperado
50 Hz	$f \ll f_c$	Atenuación fuerte
200 Hz	$f < f_c$	Atenuación significativa
480 Hz	$f = f_c$	Atenuación de -3 dB (70.7 %)
1000 Hz	$f > f_c$	Paso con mínima atenuación
2000 Hz	$f \gg f_c$	Paso sin atenuación



2.3. Filtro Pasa Banda de Segundo Orden

Parámetros

- Frecuencia central: $f_o = 580$ Hz
- Ancho de banda: $BW = 45$ Hz
- Frecuencias de corte: $f_{c1} = f_o - BW/2 = 557.5$ Hz, $f_{c2} = f_o + BW/2 = 602.5$ Hz

Se aplica pre-warping a ambas frecuencias de corte:

$$\Omega_1 = 2F_s \tan\left(\frac{\pi f_{c1}}{F_s}\right) = 16000 \tan\left(\frac{\pi \cdot 557.5}{8000}\right) = 16000 \tan(0.21893) \quad (27)$$

$$= 16000 \times 0.22237 = 3557.9 \text{ rad/s} \quad (28)$$

$$\Omega_2 = 2F_s \tan\left(\frac{\pi f_{c2}}{F_s}\right) = 16000 \tan\left(\frac{\pi \cdot 602.5}{8000}\right) = 16000 \tan(0.23658) \quad (29)$$

$$= 16000 \times 0.24203 = 3872.5 \text{ rad/s} \quad (30)$$

$$\Omega_0 = \sqrt{\Omega_1 \cdot \Omega_2} = \sqrt{3557.9 \times 3872.5} = \sqrt{13\,779\,000} \approx 3712.0 \text{ rad/s} \quad (31)$$

$$\Omega_0^2 \approx 13\,778\,944 \quad (32)$$

$$BW_w = \Omega_2 - \Omega_1 = 3872.5 - 3557.9 = 314.6 \text{ rad/s} \quad (33)$$

Función de Transferencia del Pasa Banda

El prototipo analógico Butterworth pasa banda de segundo orden:

$$H(s) = \frac{BW_w \cdot s}{s^2 + BW_w \cdot s + \Omega_0^2} \quad (34)$$

Aplicando la transformación bilineal $s = K \frac{z-1}{z+1}$ con $K = 2F_s = 16000$:

$$H(z) = \frac{BW_w \cdot K \cdot \frac{z-1}{z+1}}{\left(K \frac{z-1}{z+1}\right)^2 + BW_w \cdot K \cdot \frac{z-1}{z+1} + \Omega_0^2} \quad (35)$$



Multiplicando numerador y denominador por $(z + 1)^2$:

$$H(z) = \frac{BW_w \cdot K (z^2 - 1)}{K^2(z - 1)^2 + BW_w \cdot K (z^2 - 1) + \Omega_0^2(z + 1)^2} \quad (36)$$

2.3.1. Expansión del Denominador

$$K^2(z - 1)^2 = K^2 z^2 - 2K^2 z + K^2 \quad (37)$$

$$BW_w \cdot K (z^2 - 1) = BW_w \cdot K z^2 - BW_w \cdot K \quad (38)$$

$$\Omega_0^2(z + 1)^2 = \Omega_0^2 z^2 + 2\Omega_0^2 z + \Omega_0^2 \quad (39)$$

Sumando término a término:

$$D(z) = \underbrace{(K^2 + BW_w K + \Omega_0^2)}_{\alpha} z^2 + \underbrace{(-2K^2 + 2\Omega_0^2)}_{\beta} z + \underbrace{(K^2 - BW_w K + \Omega_0^2)}_{\gamma} \quad (40)$$

Sustituyendo valores numéricos ($K = 16000$, $K^2 = 256\,000\,000$):

$$\alpha = 256\,000\,000 + 5\,033\,600 + 13\,778\,944 = 274\,812\,544 \quad (41)$$

$$\beta = -512\,000\,000 + 27\,557\,888 = -484\,442\,112 \quad (42)$$

$$\gamma = 256\,000\,000 - 5\,033\,600 + 13\,778\,944 = 264\,745\,344 \quad (43)$$

Coefficientes del Filtro Discreto

Dividiendo numerador y denominador entre α :



$$A_0 = \frac{BW_w \cdot K}{\alpha} = \frac{314.6 \times 16000}{274\,812\,544} = \frac{5\,033\,600}{274\,812\,544} \approx 0.01832 \quad (44)$$

$$A_1 = 0 \quad (45)$$

$$A_2 = -A_0 = -0.01832 \quad (46)$$

$$B_1 = -\frac{\beta}{\alpha} = -\frac{-484\,442\,112}{274\,812\,544} = \frac{484\,442\,112}{274\,812\,544} \approx 1.76283 \quad (47)$$

$$B_2 = -\frac{\gamma}{\alpha} = -\frac{264\,745\,344}{274\,812\,544} \approx -0.96337 \quad (48)$$

Verificación

En DC ($z = 1$):

$$H(1) = \frac{A_0(1) + 0 + A_2(1)}{1 - B_1 - B_2} = \frac{0.01832 - 0.01832}{1 - 1.76283 + 0.96337} = \frac{0}{0.20054} = 0 \quad \checkmark \quad (49)$$

En Nyquist ($z = -1$):

$$H(-1) = \frac{A_0 + A_2}{1 + B_1 - B_2} = \frac{0}{1 + 1.76283 + 0.96337} = 0 \quad \checkmark \quad (50)$$

El filtro pasa banda rechaza correctamente tanto las señales DC como las de frecuencia Nyquist.

2.3.2. Función de Transferencia Final

$$H(z) = \frac{0.01832 z^2 - 0.01832}{z^2 - 1.76283 z + 0.96337} = \frac{0.01832(1 - z^{-2})}{1 - 1.76283 z^{-1} + 0.96337 z^{-2}} \quad (51)$$

Diagrama de Bloques

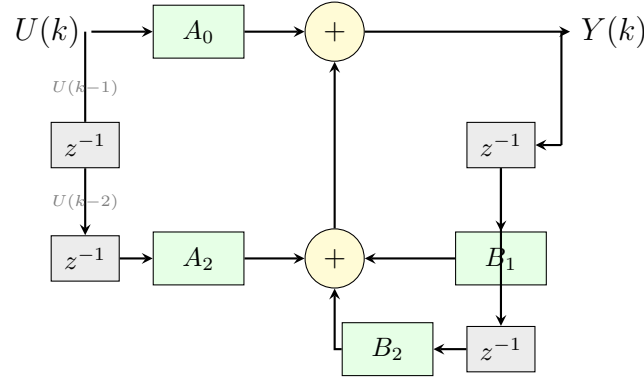


Figura 3: Diagrama de bloques del filtro pasa banda de segundo orden (Forma Directa I).
Nota: $A_1 = 0$, por lo que no hay rama para $U(k-1)$.

2.4. Coeficientes de todos los filtros;

Tabla 3: Coeficientes de los filtros IIR diseñados ($F_s = 8000$ Hz)

Coeficiente	Pasa Bajas	Pasa Altas	Pasa Banda
f_c o f_o	85 Hz	480 Hz	580 Hz
BW	—	—	45 Hz
A_0	0.03230	0.83970	0.01832
A_1	0.03230	-0.83970	0.0
A_2	0.0	0.0	-0.01832
B_1	0.93540	0.67940	1.76283
B_2	0.0	0.0	-0.96337

- **Pasa Bajas** ($f_c = 85$ Hz): Primer orden, ganancia unitaria en DC, atenuación de -3 dB en f_c . Con f_c baja, la señal de salida presenta una curva exponencial visible al filtrar ondas cuadradas. Probarlo con 50 Hz (paso con forma exponencial), 85 Hz (-3 dB) y 500 Hz (atenuación fuerte).
- **Pasa Altas** ($f_c = 480$ Hz): Primer orden, rechazo total en DC, ganancia unitaria en Nyquist. Elimina componentes de baja frecuencia, dejando solo transitorios (flancos) de la onda cuadrada. Probarlo con 50 Hz (atenuación), 480 Hz (-3 dB) y 2000 Hz (paso).

- **Pasa Banda** ($f_o = 580$ Hz, $BW = 45$ Hz): Segundo orden, selectivo en frecuencia. Extrae la componente a 580 Hz de la señal, produciendo una salida sinusoidal. Probarlo con 580 Hz (paso), 50 Hz (atenuación) y 2000 Hz (atenuación).

2.5. Diagrama de Conexión del Circuito

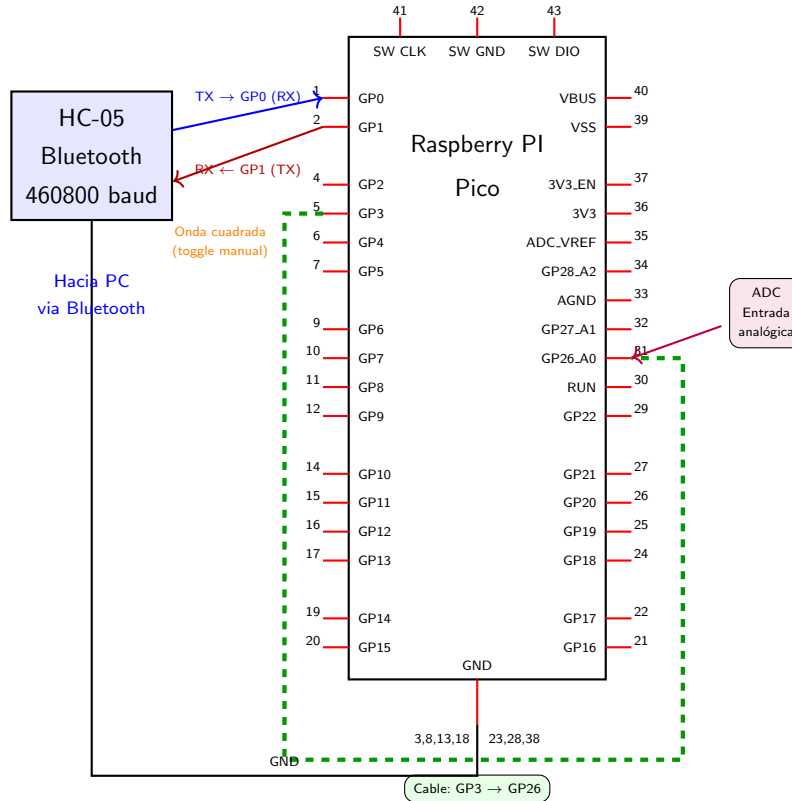


Figura 4: Diagrama de conexión del circuito: Raspberry Pi Pico 2W con módulo HC-05 y lazo de retroalimentación.

Funcionamiento del circuito:

1. MATLAB envía los coeficientes del filtro (A_0, A_1, A_2, B_1, B_2) y la frecuencia de la señal de prueba a través de Bluetooth.
2. La Pico configura la frecuencia de la onda cuadrada generada por toggle manual en el hilo del Core 2.
3. Al recibir el comando de captura (0x52), el Core 1 ejecuta la ecuación en diferencias a



8 kHz durante 500 muestras.

4. La onda cuadrada generada en el Pin 3 se conecta externamente al Pin 26 (ADC), sirviendo como señal de entrada.
5. Los vectores de entrada $U(k)$ y salida $Y(k)$ se envían de vuelta a MATLAB para su visualización.

2.6. Protocolo de Comunicación UART

La comunicación entre MATLAB y la Raspberry Pi Pico se realiza mediante UART a 460800 baudios a través del módulo HC-05. Se definen dos tipos de tramas:

MATLAB envía una trama de 25 bytes con la siguiente estructura:

Tabla 4: Estructura de la trama de coeficientes

Byte(s)	Campo	Formato	Descripción
0	Encabezado	0x55 (“U”)	Identificador de trama
1–4	A_0	float32 LE	Coeficiente de $U(k)$
5–8	A_1	float32 LE	Coeficiente de $U(k-1)$
9–12	A_2	float32 LE	Coeficiente de $U(k-2)$
13–16	B_1	float32 LE	Coeficiente de $Y(k-1)$
17–20	B_2	float32 LE	Coeficiente de $Y(k-2)$
21–24	FREQ	float32 LE	Frecuencia de prueba (Hz)

MATLAB envía el byte 0x52 para iniciar la captura de datos.

2.7. Explicación del Código:

Listing 1: Código MicroPython del simulador de filtros IIR

```
1 import machine
2 machine.freq(300_000_000) # Overclock a 300 MHz
3
4 from machine import Pin, ADC, UART
5 from time import sleep_ms, sleep_us, ticks_us, ticks_diff, ticks_add
6 import _thread
```



```
7 import struct
8
9 ADC0 = ADC(Pin(26))
10 Pin3 = Pin(3, Pin.OUT)
11 UART0 = UART(0, baudrate=460800, tx=Pin(0), rx=Pin(1))
12
13 conversion_factor = 3.3 / 65535
14 MUESTRAS = 1000
15 ENTRADAS = [0]*MUESTRAS
16 SALIDAS = [0]*MUESTRAS
17 En_Bytes = bytearray(4000)
18
19 CTE_A0, CTE_A1, CTE_A2 = 0.0303, 0.0303, 0.0
20 CTE_B1, CTE_B2 = 0.9394, 0.0
21 FREQ = 50
22 PERIODO = 1/FREQ
23 HALF = int((PERIODO*1000000)/2)
24
25 def Onda_Cuadrada():
26     while True:
27         Pin3.on()
28         sleep_us(HALF)
29         Pin3.off()
30         sleep_us(HALF)
31
32 @micropython.native
33 def capturar(a0, a1, a2, b1, b2):
34     uk = uk1 = uk2 = 0.0
35     yk = yk1 = yk2 = 0.0
36     adc = ADC0.read_u16
37     cf = conversion_factor
38     ent = ENTRADAS
39     sal = SALIDAS
40     _tu = ticks_us
41     _td = ticks_diff
42     _ta = ticks_add
```



```
43     _su = sleep_us
44
45     t_next = _tu()
46     for i in range(500):
47         yk = a0*uk + a1*uk1 + a2*uk2 + b1*yk1 + b2*yk2
48         uk2 = uk1
49         uk1 = uk
50         yk2 = yk1
51         yk1 = yk
52         uk = adc() * cf
53         ent[i] = uk
54         sal[i] = yk
55         t_next = _ta(t_next, 125)
56         dt = _td(t_next, _tu())
57         if dt > 0:
58             _su(dt)
59
60 _thread.start_new_thread(Onda_Cuadrada, ())
61
62 while True:
63     if UART0.any() > 0:
64         sleep_ms(100)
65         LLAVE = UART0.read()
66         if LLAVE and LLAVE[0] == 0x55:
67             # Decodificar 6 flotantes IEEE 754 LE
68             CTE_A0 = struct.unpack('<f', LLAVE[1:5])[0]
69             CTE_A1 = struct.unpack('<f', LLAVE[5:9])[0]
70             CTE_A2 = struct.unpack('<f', LLAVE[9:13])[0]
71             CTE_B1 = struct.unpack('<f', LLAVE[13:17])[0]
72             CTE_B2 = struct.unpack('<f', LLAVE[17:21])[0]
73             FREQ = struct.unpack('<f', LLAVE[21:25])[0]
74             HALF = int((1/FREQ * 1000000) / 2)
75         if LLAVE and 0x52 in LLAVE:
76             capturar(CTE_A0, CTE_A1, CTE_A2, CTE_B1, CTE_B2)
77             for i in range(500):
78                 struct.pack_into('f', En_Bytes, i*8, ENTRADAS[i])
```



```
79         struct.pack_into('f', En_Bytes, i*8+4, SALIDAS[i])
80     for j in range(0, 4000, 50):
81         UART0.write(En_Bytes[j:j+50])
82         sleep_ms(60)
```



3. Resultados:

3.1. Filtro Pasa Bajas ($f_c = 85 \text{ Hz}$)

3.2. Filtro Pasa Altas ($f_c = 480 \text{ Hz}$)

3.3. Filtro Pasa Banda ($f_o = 580 \text{ Hz}$, $BW = 45 \text{ Hz}$)



4. Conclusiones:

- Espinoza Matamoros Percival Ulises:
- Flores Colin Victor Jaziel:
- García Cortés Adolfo de Jesús:
- Lara Hernandez Angel Husiel:
- Lugo Manzano Rodrigo:



Referencias

- Butterworth, S. (1930). On the Theory of Filter Amplifiers. *Wireless Engineer*, 7(6), 536-541.
- MicroPython Contributors. (2024a). *_thread — Multithreading Support*. https://docs.micropython.org/en/latest/library/_thread.html
- MicroPython Contributors. (2024b). *class ADC — Analog to Digital Conversion*. <https://docs.micropython.org/en/latest/library/machine.ADC.html>
- MicroPython Contributors. (2024c). *class PWM — Pulse Width Modulation*. <https://docs.micropython.org/en/latest/library/machine.PWM.html>
- MicroPython Contributors. (2024d). *machine — Functions Related to the Hardware*. <https://docs.micropython.org/en/latest/library/machine.html>
- MicroPython Contributors. (2024e). *time — Time Related Functions*. <https://docs.micropython.org/en/latest/library/time.html>
- Ogata, K. (2010). *Modern Control Engineering* (5.^a ed.). Pearson.
- Python Software Foundation. (2024). *_thread — Low-Level Threading API*. https://docs.python.org/3/library/_thread.html
- Raspberry Pi Ltd. (2024a). *Raspberry Pi Pico 2 W Datasheet*. <https://datasheets.raspberrypi.com/pico/pico-2-w-datasheet.pdf>
- Raspberry Pi Ltd. (2024b). *RP2350 Datasheet: A Microcontroller by Raspberry Pi*. <https://datasheets.raspberrypi.com/rp2350/rp2350-datasheet.pdf>
- Sedra, A. S., Smith, K. C., Carusone, T. C., & Gaudet, V. (2020). *Microelectronic Circuits* (8.^a ed.). Oxford University Press.